SignSense: A Filipino Sign Language (FSL) Detector

Far Eastern University - Manila

In Fulfillment of the Requirements in 3rd Year DSC1101- Introduction to Data Science

Mx. Richmond Lim
Facilitator

Naomi Hannah A. Cuerdo
Jade Marco S. Morillo
Kyte Daiter M. Percia

December 06, 2024

I. Objectives

SignSense is a system designed to detect Filipino Sign Language (FSL) using Python and Machine Learning, specifically Tensorflow, Keras, and OpenCV 3. This project aims to bridge the communication gap between hearing and deaf communities, making everyday interactions more accessible. SignSense aims to promote inclusivity and facilitate communication in public spaces, school and work places, and beyond.

Additionally, SignSense addresses the unique challenges of Filipino Sign Language, which is distinct from other sign language, as existing solutions often lack coverage for its specific gestures and expressions. This can be a foundation for paving the way for future advancements in assistive technology for the deaf and mute community.

With this, SignSense aims to achieve the following:

1. **Develop a real-time sign language recognition system** capable of detecting and translating FSL gestures.
2. **Promote accessibility and inclusivity** by creating a safe and supportive communication environment in different settings.
3. **Develop a user-friendly interface** that can be accessed in multiple platforms, such as mobile and desktop.

II. Tools and Technologies Used

To implement SignSense, **Python 3** was used as a programming language for this project. The developers also used **Tensorflow, Keras, and OpenCV 3.** Each tool had different roles: OpenCV 3 was used to handle real-time video and image processing, specifically, converting the images into grayscale to simplify data for the model; TensorFlow backed up the deep learning for training the model; and Keras was used to simplify the development and training of the Conventional Neural Network (CNN) which was the heart of this project, responsible for recognizing and classifying hand gestures.

III. Methodology

1. Data Collection

Due to hardware limitations, the developers captured only 11 letters of the alphabet, namely: A, C, E, I, K, M, N, O, R, T, and Y. The chosen letters were derived from the developers' preferred names and were used to create the prototype of **SignSense**. A total of **two-hundred fifty (250) images per letter** were collected to form the dataset. These images were then processed and converted into **grayscale** using OpenCV 3 to simplify the data and reduce

complexity. The images were then split into a **training set** and a **test set** for model evaluation.

2. Training the Model

After data was collected, the developers then compiled and started training the model using a **Convolutional Neural Network (CNN).** The model was structured using Keras with the help of TensorFlow as backend. The training process was as follows:

1. Model Architecture

The CNN was initialized as a sequential model, starting with an input layer of accepting images of shape **64x64x3** (height, width, and 3 color channels for RGB images). **Three convolutional layer**s were added to the model, with the first layer having **32 filters** with a kernel size of 3x3, followed by introducing non-linearity using **ReLU activation**, and a **BatchNormalization** for stabilizing the model process. The additional layers, each with **64 and 128 filters respectively**, followed the same pattern while introducing **MaxPooling2D** to down-sample the feature maps and reduce complexity.

2. Regularization and Dropout

To avoid overfitting and large weights, **L2 regularization** was applied to the convolution layers. A dropout of **0.5** was applied after to avoid overfitting and ensure that the model works.

3. Fully Connected Layers

After the convolutional layers, the data was flattened to a 1D vector and passed through a **Dense** layer with **512 neurons and ReLU activation.** The output had 11 neurons, one for each letter, with a softmax activation function to produce a probability distribution over the 11 classes.

4. Model Compilation

The model was compiled using the **Adam Optimizer** with a learning rate of 0.0005, suitable for the dataset and complexity of the task. A **categorical cross-entropy loss function** was then used to track its progress using accuracy.

5. **Data Augmentation**

   To improve generalization, data augmentation was implemented with techniques such as shearing, zooming, horizontal flipping, and rotation (a range of 30 degrees) to increase the dataset's variety and size.

6. **Callbacks**

   A couple of callbacks were introduced to track the model's accuracy. **EarlyStopping** was used to halt training if the validation loss did not improve for 5 epochs, restoring the model to its best weights. Additionally, **ReduceLROnPlateau** was also used to reduce the learning rate if the validation loss stopped improving to ensure that the model converges efficiently.

7. **Training Proper**

   The model was trained for **50 epochs,** with a batch size of 64 images per iteration. The training process used the augmented training data, and the performance was monitored using the test set. The training and validation accuracy were plotted to visualize the model's learning process.

IV. Application Output

1. Website Prototype

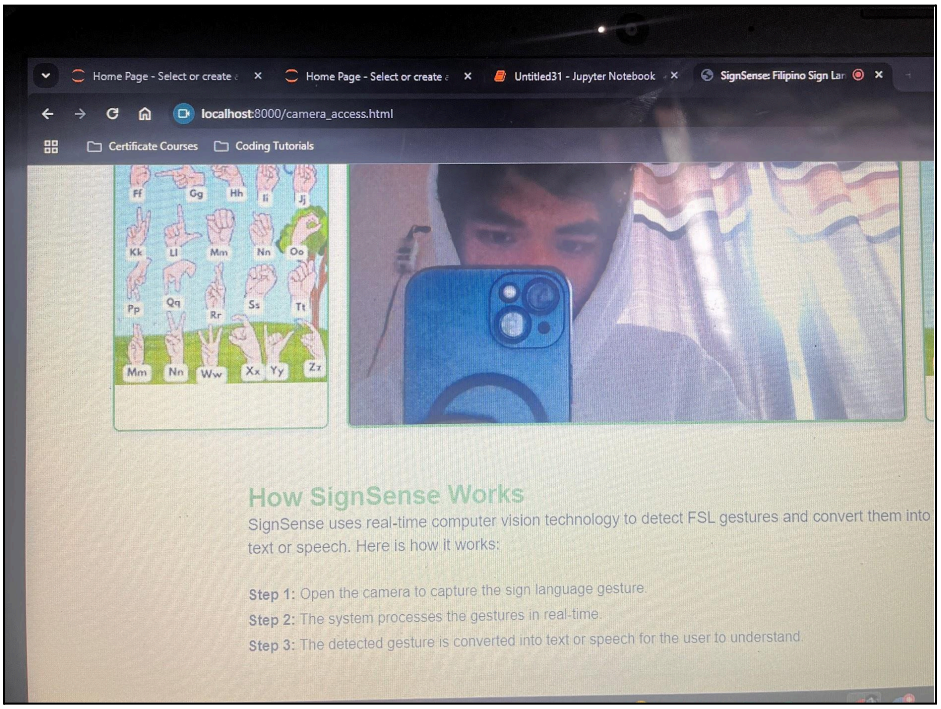

Figure 1. Website Prototype

Figure 1 illustrates the developers' website prototype, showcasing the planned integration of SignSense. The website will feature a dedicated camera detection interface, enabling users to interact directly with the SignSense technology. Additionally, it will include a comprehensive guide to the Filipino Sign Language (FSL) alphabet, along with concise instructions on how SignSense operates. By centralizing these features, the website reflects SignSense's goal of developing a real-time sign language recognition system that promotes accessibility and inclusivity through a user-friendly interface, in this case, a website.

2. Training Model Accuracy and Loss

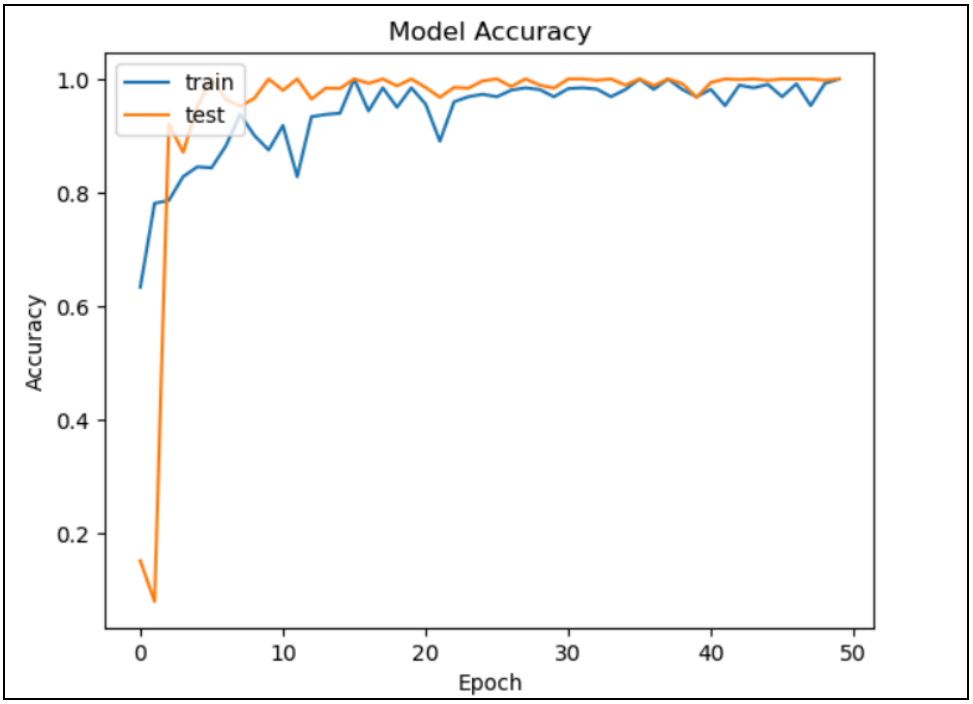

Figure 1. Training and Testing Model Accuracy of the SignSense

Figure 1 shows the training accuracy of the model for Signsense across 50 epochs. The training and testing accuracy started low and it then rapidly increased, reaching close to a 100% accuracy within the first 10 epochs. Both metrics stabilize with minimal fluctuation, indicating that the model performs well on both training and testing data, demonstrating good generalization. The close alignment between the two curves (training and test) also suggests minimal overfitting.

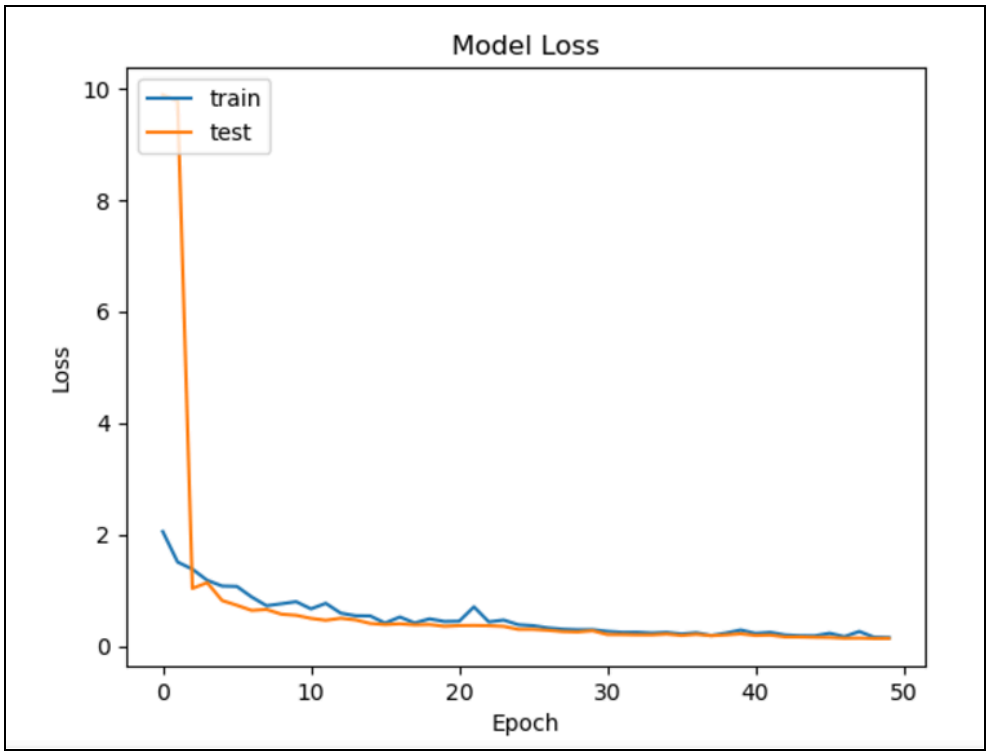

Figure 2. Training Model Loss for SignSense

Figure 2 shows the loss during training and testing across epochs. Both training and testing loss decrease during the first 10 epochs, which indicates that the model is quickly learning the patterns in the data. The loss stabilizes after the drop, suggesting that the model has learned as much as it can from the data set. Both curves are very close to each other throughout the training process, implying that there is minimal overfitting, a good indication that the model is performing similarly on both the training and testing data sets.

3. Application Output

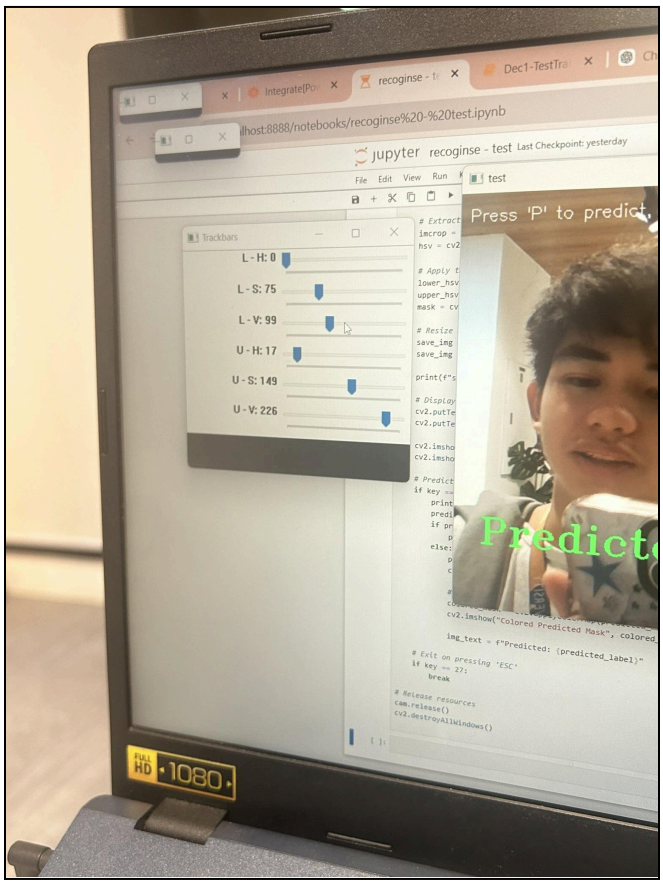

Figure 3. Testing the Hue Trackbars

Figure 3 shows the trackbars for adjusting the gray scale during the real-time testing of SignSense.

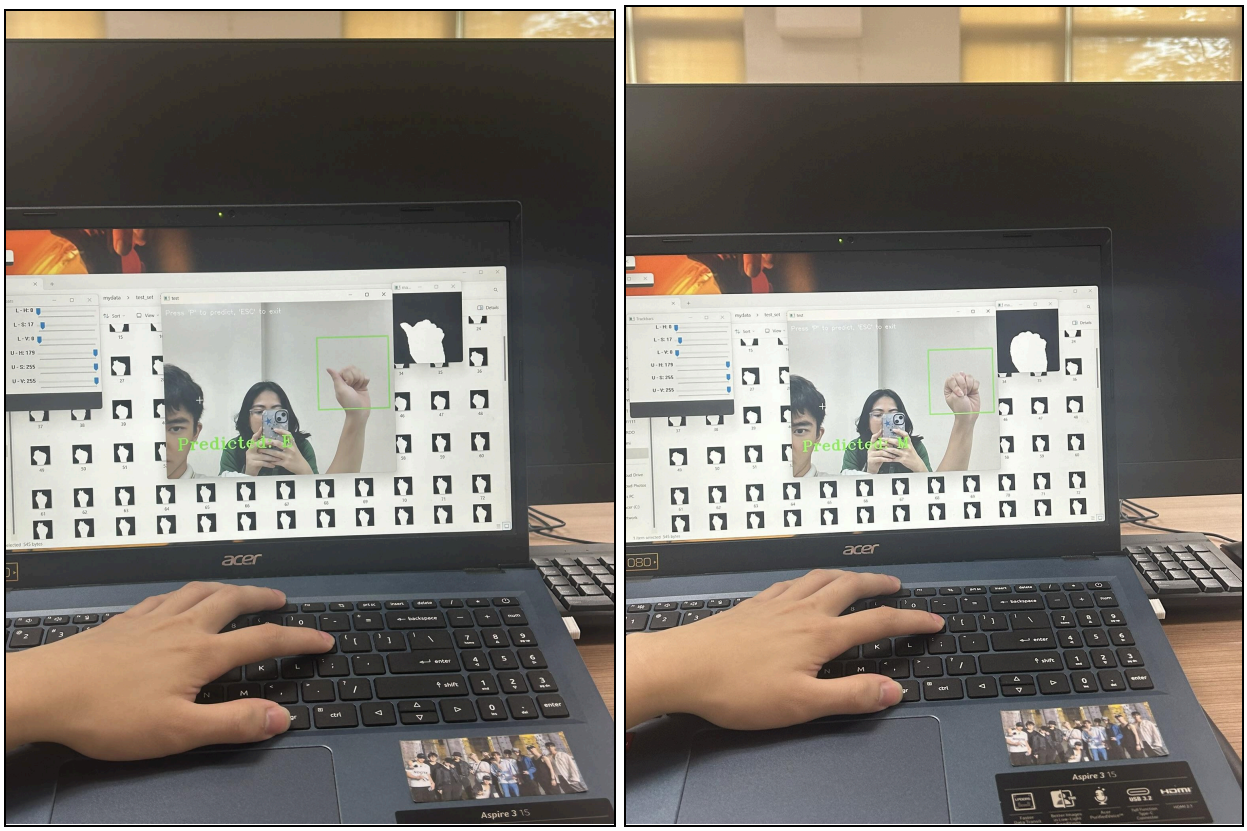

Figure 5 and Figure 6. Testing the Model

Figures 5 and 6 show a picture of the developers testing the model. From the figure, it shows that the model can read the letter done on the green box by converting it to a grayscale for better accuracy.

V. Summary

SignSense successfully developed a real-time Filipino Sign Language (FSL) detection system, capable of recognizing and translating 11 key FSL letters using machine learning techniques such as TensorFlow, Keras, and OpenCV 3. The primary objective was to create a real-time sign language recognition system that could promote inclusivity and bridge the communication gap between the deaf and mute.

By training a Convolutional Neural Network (CNN) on a dataset of 2,750 images representing 11 FSL letters, the system achieved high accuracy and minimal overfitting, which meets the goal of effective gesture detection.

Furthermore, the project provided an interactive user interface that allows the user to adjust the grayscale for real-time testing and recognition, making communication more accessible. Despite hardware limitations that restricted the developers to create a data set of only 11 letters, the project still has a strong potential for future improvements, such as the inclusion of letters “Ñ” and “NG”, and enhancing the system’s capabilities to handle real-time object detection without using grayscale processing.

Thus, the objectives of SignSense were effectively met by creating a fully functional model, implementing real time capture of the gestures, and setting a foundation for future enhancements that would further enhance communication among the deaf and mute communities.